COMP2321 Artificial Intelligence

Date: 05/11/2025

Version: 1

Coursework: Solving Classical AI Problems using Search, Reasoning, and Game-Playing Techniques

Group Members-1: Low Yee Heng Kevin, yhkl1e24@soton.ac.uk

School of Computer Science

University of Southampton Malaysia

Table of Contents

Task 1: Problem Definition and Environment Design

Task 1.1: Problem formalisation

Problem Definition

Problem statement:
The goal of tic tac toe is for a player to place three of their (either ‘X’, or ‘O’) in a row –horizontally, vertically or diagonally before the opponent does. If all nine squares are filled without either player achieving three in a row, the game ends in a draw.

- State Representation: A state is a tuple (board, player_to_move)

  board: A 3 x 3 grid. Each cell contains either X, O, or EMPTY.

  player_to_move: The player (X or O) whose turn it is to move

- Initial State:

  board: All nine cells are EMPTY.

  player_to_move: X (X moves first)
- Goal State: Goal state occurs when game has ended.

  Win for X: the board has 3 X’s in any row, column or diagonal.

  Win for O: the board has 3 O’s in any row, column or diagonal.

  Draw or Tie: All nine cells are full and neither player has won.

Actions and Transition model

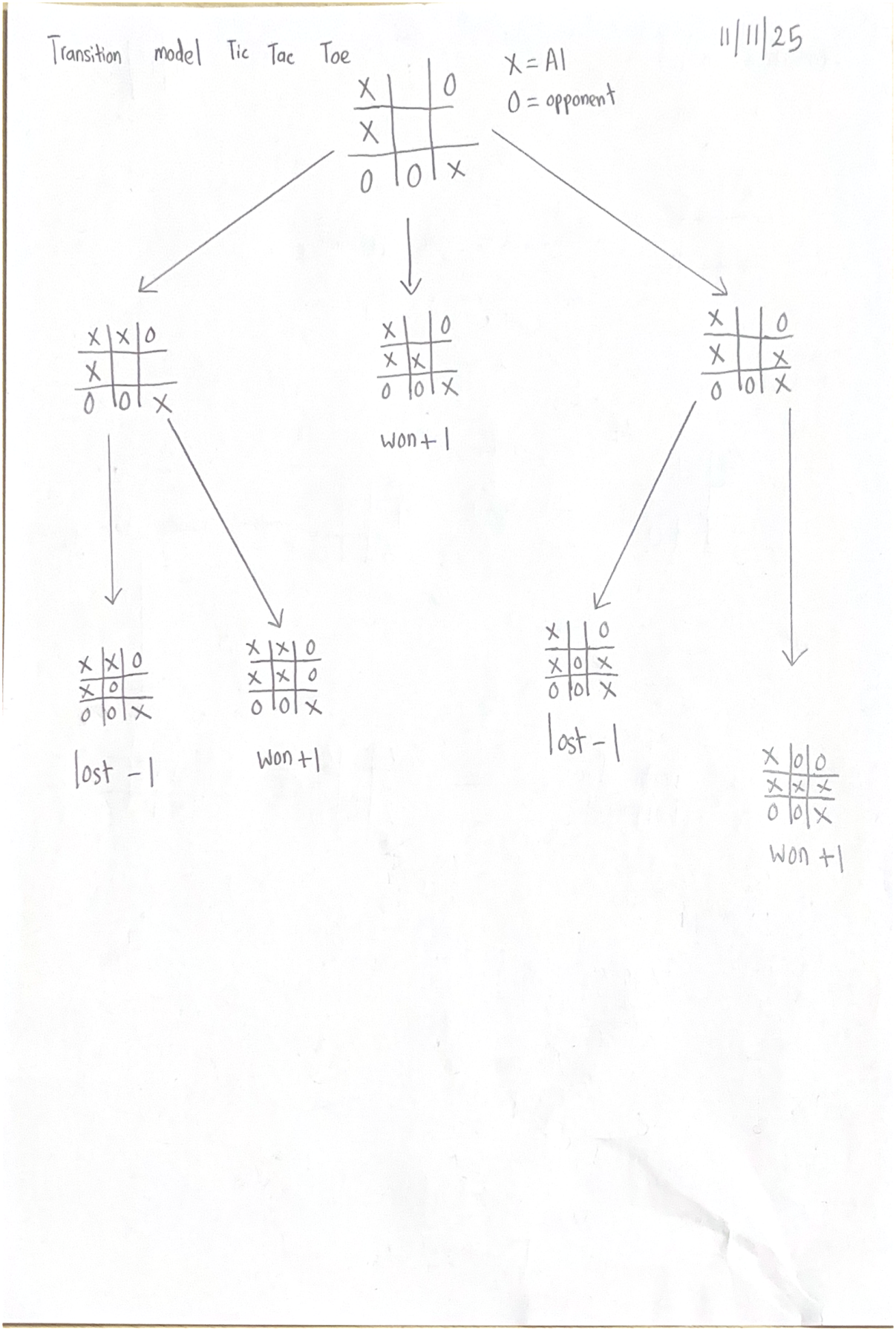
- Actions: The set of available actions in a state s = (board, player_to_move)

  An action is defined as Place(r,c), where r = row and c = column, both in {0,1,2}

  The set of legal actions are: Actions(s) = {Place(r,c) | s.board[r][c] == EMPTY}

  In total number of steps and possibilities per each state

Figure 1: Transition Model



Transition model = Result(s,a) that defines new state s' that results in acting a in state s.

Let current state s = (B, P), where B is the 3x3 board and P is the player_to_move

Let action a = Place(r, c).

The new state s' = (B', P') is defined by:

$$B'[i][j] = \begin{cases} P & \text{if } i = r \text{ and } j = c \\ B[i][j] & \text{otherwise} \end{cases}$$

P' is the opponent of the current player P. This equation ensures that the turn alternates after every move.

P' = Opponent(P)

The possibilities equation = BF(s)

Actions(s) = {Place(r,c) | s.board[r][c] == EMPTY}

BF(s) = |Actions(s)|

If k is the step number of total number of marks currently on board, k = 0 for initial state, the branching factor is:

BF(s) = 9 - k

This shows that the number of possibilities decreases by exactly one at every step of the game.

Constraints

- Move Legality: A player can only place mark in a cell that is EMPTY.
- Turn-based: Players must alternate turns.
- Game end: No further actions can be taken once goal state is reached.

Conditions for winning

A player P wins if they have three of their mark in any row, column, or diagonal.

Function Win(B, P) which returns TRUE if player P has won on board B.

1. Row wins
   A player P wins in row i:

$$\text{Row}(B, P, i) = \bigwedge_{j=0}^{2} (B(i, j) = P)$$

This means B(i, 0) = P AND B(i, 1) = P AND B (i, 2) = P.

2. Column Wins
   A player P wins in column j:

$$\text{Col}(B, P, j) = \bigwedge_{i=0}^{2} (B(i, j) = P)$$

   This means B(0, j) = P AND B(1, j) = P AND B (2, j) = P.

3. Diagonal wins
   A player P wins on the main diagonal

$$\text{Diag}_1(B, P) = \bigwedge_{i=0}^{2} (B(i, i) = P)$$

   This means B (0, 0) = P AND B (1, 1) = P AND B(2, 2) = P

   A player P wins on the anti-diagonal

$$\text{Diag}_2(B, P) = \bigwedge_{i=0}^{2} (B(i, 2 - i) = P)$$

   This means B (0, 2) = P AND B (1, 1) = P AND B (2, 0) = P

Final win equation

$$\text{Win}(B, P) = \left(\bigvee_{i=0}^{2} \text{Row}(B, P, i) \right) \vee \left(\bigvee_{j=0}^{2} \text{Col}(B, P, j) \right) \vee \text{Diag}_1(B, P) \vee \text{Diag}_2(B, P)$$

Environment characterisation

- Deterministic: The outcome of every action is perfectly defined. No chance in this game. The next state of the environment is completely dependent on the current state and the action executed.
- Fully observable: Able to see all the states of the environment. No hidden information.
- Discrete: Yes. Finite number of board states and finite set of actions
- Multi-agent: As there are two agents in the game.

Task 1.2: Environment diagram

Figure 2: Empty grid

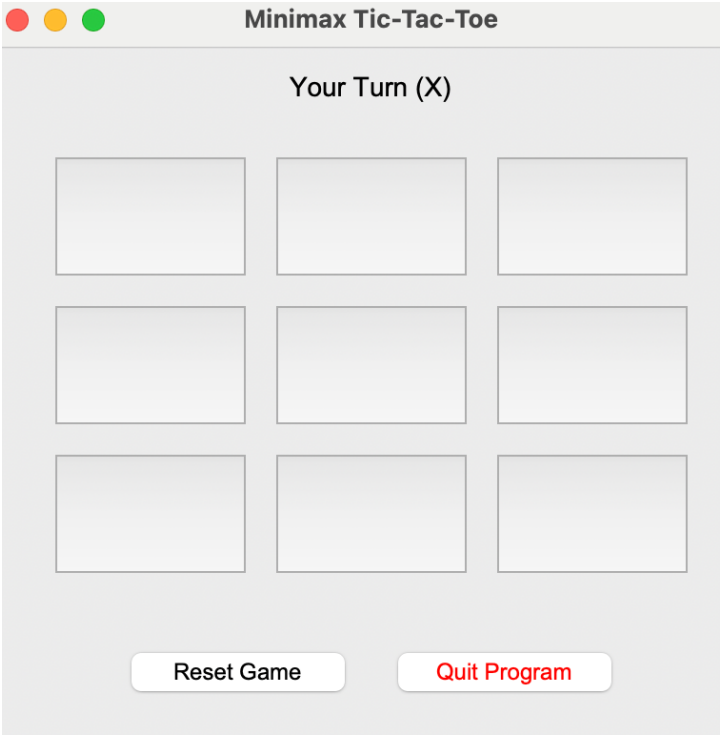


Figure 3: Empty grid in GUI

Task 1.3: Test instances and assumptions

Test instances or benchmarks

- Opening Moves:

  As player X (first move), must choose a corner.

  As player O (second move), must choose a corner (if available, otherwise block)

- Forcing win: The agent must prioritise any move that results in immediate win. The agent must play to win and ignore opponent's threat as its own win is immediate.

- Blocking loss: If no immediate win is possible then agent must block any move that would allow opponent to win on next turn.

- Fork creation: Agent should create a "fork" (two simultaneous winning threats) when possible so that the opponent cannot block at the same time.

- Full game: In a "perfect" scenario, the game must end in a draw, this is to validate the entire Minimax algorithm implementation.

Assumptions and simplifications

- Utility function: Agent win: +1, Agent loss: -1, Agent draw: 0
- Scope: Simplify to only 3x3 grid.
- Opponent: The agent assumes it is playing against a "perfect" opponent.
- Start player: The game always begins with player X

Task 2: Algorithm Selection and Design

Task 2.1: Algorithm selection and justification.

Chosen algorithm: Minimax algorithm

Minimax is a recursive and depth first search algorithm which is used in decision-making and game theory. It provides an optimal move for the player assuming that opponent is also playing optimally. It operates by building a game tree from the current state down to all possible terminal states.

Minimax algorithm is most suitable because the game's characteristics fit with the algorithm's assumptions.

1. Two-player, Zero-sum: Tic tac toe is a 2 player game. A win for X is a direct loss for 0, and a draw is a neutral outcome for both.
2. Perfect information: The entire game board is always visible to both players. No hidden information.
3. Deterministic: Every action has a single predictable outcome with no chance involved.
4. Finite state space: The game has a small and finite game tree. There are only 9 cells, hence the total number of reachable, unique states are ~9! = 362880.

Since game tree is small, minimax can find an optimal solution every time.

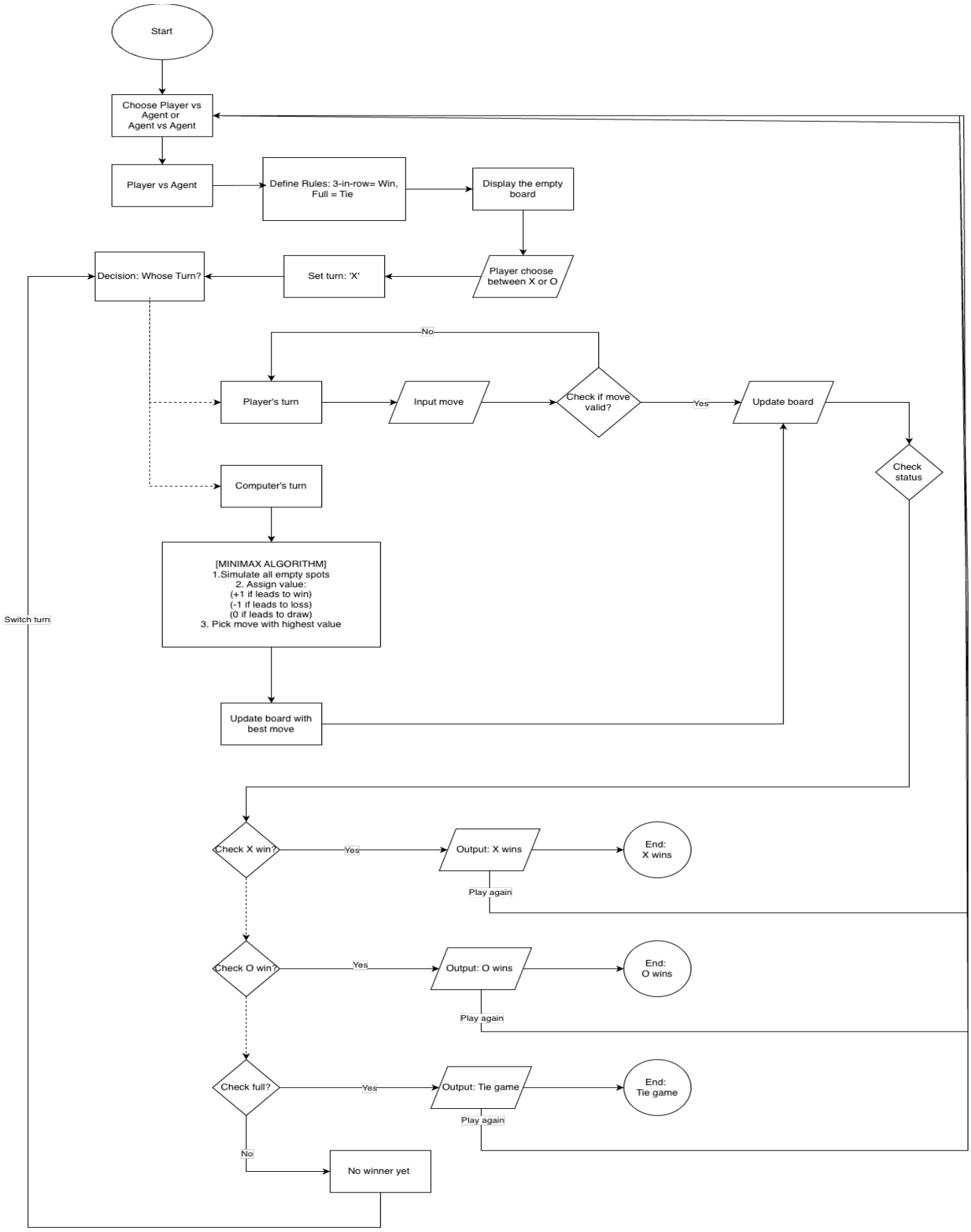
Task 2.2: Algorithm design and structure

```
function MINIMAX (board, current_player, MAX_PLAYER, MIN_PLAYER) returns a utility value:
    INCREMENT nodes_explored
    if IS-WINNER(board, MIN_PLAYER) then
        return -1
    else if IS-WINNER(board, MAX_PLAYER) then
        return +1
    else if IS-BOARD-FULL(board) then
        return 0

    if current_player == MAX_PLAYER then
        best_score = -∞

        for each move in AVAILABLE-MOVES(board) do
            new_board = APPLY-MOVE(board, move, MAX_PLAYER)
            score = MINIMAX(new_board, MIN_PLAYER, MAX_PLAYER, MIN_PLAYER)
            best_score = MAX(best_score, score)
        return best_score
    else
        best_score = +∞
        for each move in AVAILABLE-MOVES(board) do
            new_board =APPLY-MOVE(board, move, MIN-PLAYER)
            score = MINIMAX(new_board, MAX_PLAYER, MAX_PLAYER, MIN_PLAYER)
            best_score = MIN(best_score, score)
        return best_score
```

Figure 4: Flowchart for the game



Task 2.3: Heuristics or optimisation parameters

Evaluation function

The algorithm's decision making is guided by a utility function. This function only applied at terminal nodes of the game tree.

- Utility(state) = +1: If state is a win for the agent(MAX)
- Utility(state) = -1: If state is a loss for agent(MIN wins)
- Utility(state) = 0: State is a draw

Heuristics

Heuristic rules are not required because the state space is sufficiently small to allow a full search to the end of the game. Relying on rules is unnecessary as they do not allow the agent to look ahead and verify the strategic consequences of a move. Instead, the algorithm will search until a terminal state is reached, assigning a precise utility value to every outcome: +1 for a win (good move), 0 for a draw, and -1 for a loss (bad move).

Parameters and limits

Depth limit: None. The algorithm is designed to search full depth of the game. This is possible due to the small, finite state space.

Utility values: The values +1, -1, 0 are the only parameters. These are fixed and sufficient to represent the zero-sum nature of the game.

Completeness, optimality and efficiency

- Completeness: The algorithm ensures completeness, meaning it is guaranteed to achieve its target of identifying the optimal move. Since the Tic-Tac-Toe game is finite and free of infinite loops, the depth first search strategy of the minimax algorithm will always reach a terminal state and return a valid utility value.

- Optimality: The algorithm is optimal. It will find the “perfect” move, assuming the opponent also plays optimally. For tic-tac-toe, this means the agent will always win if a win is possible and draw if a win is not possible.

- Efficiency: The algorithm demonstrates high performance as the agent computes optimal moves with zero perceptible lantency. Despite exponenetial time complexity, the small finite state space allows for millisecond execution, while the depth first search ensures minimal memory overhead.

Handling of invalid states and tie breaking

Invalid states: The algorithm’s logic already handles this because ACTIONS(state) only returns actions for EMPTY cells.

Tie-breaking: The arg max function in MINIMAX-DECISION handles tie breaking. If 2 or more moves lead to the same value, it will simply select the first one found in the loop.

Task 3: Implementation and Experimental Results

The Tic-Tac-Toe game was implemented using Python 3.13. Python was selected for its readability and efficient handling of logic and recursion, which are essential for implementing the Minimax algorithm. The IDE used to write, test, and debug the code was PyCharm. PyCharm provided necessary tools for syntax highlighting, code structure analysis, and a built-in terminal for running the text-based simulation and performance tests.

Task 3.1: Implementation accuracy

Figure 5: Minimax Function

```
def minimax(board, current_player_letter, ai_letter, ai_opponent_letter):

    global nodes_explored
    nodes_explored += 1

    if isWinner(board, ai_opponent_letter):
        return -1
    elif isWinner(board, ai_letter):
        return 1
    elif isBoardFull(board):
        return 0


    if current_player_letter == ai_letter:
        best_score = -float('inf')
        for move in range(1, 10):
            if isSpaceFree(board, move):
                boardCopy = getBoardCopy(board)
                makeMove(boardCopy, current_player_letter, move)
                score = minimax(boardCopy, ai_opponent_letter, ai_letter, ai_opponent_letter)
                best_score = max(best_score, score)
        return best_score

    else:
        best_score = float('inf')
        for move in range(1, 10):
            if isSpaceFree(board, move):
                boardCopy = getBoardCopy(board)
                makeMove(boardCopy, current_player_letter, move)
                score = minimax(boardCopy, ai_letter, ai_letter, ai_opponent_letter)
                best_score = min(best_score, score)
        return best_score
```

This is the core recursive function. It implements a depth-first search to explore all possible game states. It identifies terminal states of win, loss and draw and according utility values (+1 for win, 0 for draw, -1 for loss). It alternates the logic based on the current_player to simulate “Min” and “Max” perspectives.

The agent assigns a specific numerical value to every possible ending of the game. This is found in the minimax function’s base cases. In best_score = min(best_score, score) the agent knows a move is bad if it allows the opponent to force a win. It knows this because inside the minimax function, it simulates the opponent’s turn using this function.

Figure 6: Determine agent's move

```
def getComputerMove(board, computerLetter):

    global nodes_explored
    nodes_explored = 0

    if computerLetter == 'X':
        playerLetter = 'O'
    else:
        playerLetter = 'X'

    best_score = -float('inf')
    best_move = None

    for move in range(1, 10):
        if isSpaceFree(board, move):
            boardCopy = getBoardCopy(board)
            makeMove(boardCopy, computerLetter, move)


            score = minimax(boardCopy, playerLetter, computerLetter, playerLetter)

            if score > best_score:
                best_score = score
                best_move = move

    return best_move
```

This is the function for the agent. It iterates through the immediate set of legal moves and calls minimax for each and selects the action with the highest utility returned. The actual decision for choosing moves happens here. The agent starts by assuming the worst possible score -∞ and looks for anything better.

For example, the agent simulates Move A. The minimax returns a 0. It evaluates and since 0 is bigger than -∞. The agent will know that move A is the current best option. The agent then simulates move B. The minimax function returns a -1. Since -1 is smaller than 0. The agent knows that move B is worse than move A and will ignore move B. Minimax then simulates move C and minimax returns a 1. Since 1 is bigger than 0, the agent knows that move C is better than move A and decide that move C is the new best option. After checking all moves, it returns best_move.

Figure 7: Simulation between 2 agents

```
def runAIGames(num_games):
    print(f"Starting {num_games} AI-vs-AI games...")
    start_time = time.time()
    stats = {'X': 0, 'O': 0, 'Draw': 0}
    total_simulation_nodes = 0
    if num_games < 10:
        print_interval = 1
    else:
        print_interval = num_games // 10
    for i in range(num_games):
        theBoard = [' '] * 10
        turn = 'X'
        while True:
            if turn == 'X':
                move = getComputerMove(theBoard, computerLetter: 'X')
                total_simulation_nodes += nodes_explored
                makeMove(theBoard, letter: 'X', move)
                if isWinner(theBoard, le: 'X'):
                    stats['X'] += 1
                    break
                turn = 'O'
            else:
                move = getComputerMove(theBoard, computerLetter: 'O')
                total_simulation_nodes += nodes_explored
                makeMove(theBoard, letter: 'O', move)
                if isWinner(theBoard, le: 'O'):
                    stats['O'] += 1
                    break
                turn = 'X'
            if isBoardFull(theBoard):
                stats['Draw'] += 1
                break
        if (i + 1) % print_interval == 0:
            print(f"  ...Game {i + 1}/{num_games} completed.")
    end_time = time.time()
    total_time = end_time - start_time
    print("\n--- Simulation Complete ---")
    print(f"Total Games: {num_games}")
    print(f"AI 'X' Wins: {stats['X']}")
    print(f"AI 'O' Wins: {stats['O']}")
    print(f"Draws:       {stats['Draw']}")
    print(f"Total Nodes Explored: {total_simulation_nodes:,}")
    print(f"Total time taken: {total_time:.4f} seconds")
```

This is a simulation harness developed to verify implementation correctness. It automates matches between two instances of the minimax agent to prove optimality. By facilitating a self-play experiment, the function validates that the agent adheres to game-theoretic constraints. Since tic tac toe is a zero-sum game with perfect information, two optimal agents will always result in a draw. Any deviation from this outcome would instantly signal a flaw in the recursive logic or utility evaluation. Furthermore, this function shows performance metrics to quantify the computational cost of the algorithm.

Figure 8: Win or loss verification

```
def isWinner(bo, le):
    return ((bo[1] == le and bo[2] == le and bo[3] == le) or
            (bo[4] == le and bo[5] == le and bo[6] == le) or
            (bo[7] == le and bo[8] == le and bo[9] == le) or
            (bo[1] == le and bo[4] == le and bo[7] == le) or
            (bo[2] == le and bo[5] == le and bo[8] == le) or
            (bo[3] == le and bo[6] == le and bo[9] == le) or
            (bo[1] == le and bo[5] == le and bo[9] == le) or
            (bo[3] == le and bo[5] == le and bo[7] == le))
```

This function verifies the win/loss condition. It performs a Boolean check across all 8 possible winning configurations to determine if the specified player has achieved victory. In the context of the game tree, if this returns true, the current node is a terminal state with a utility of +1 or -1.

Figure 9: Draw verification

```
def isBoardFull(board):
    for i in range(1, 10):
        if isSpaceFree(board, i):
            return False
    return True
```

This function verifies the draw condition. It iterates through the board to check for available moves. If the board is full and isWinner returns false, the game is declared a tie. In the game tree, this represents a terminal state with a utility of 0.

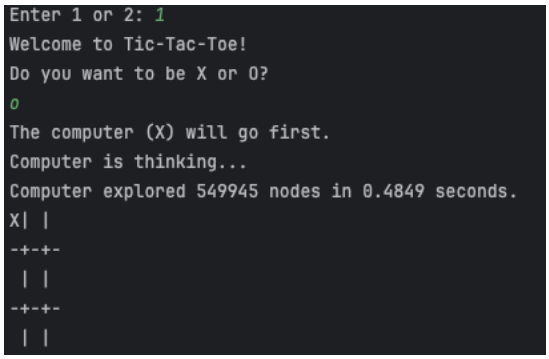
Figure 10: Nodes explored counter

```
nodes_explored = 0
```

A nodes explored counter is integrated to track computational effort for performance analysis. It is reset to 0 at the initiation of every decision cycle and incremented by one at the start of every recursive minimax call. This design makes it easier to see the size of the search space for any specific board state.

Task 3.2: Experimental results

Figure 11: Starting grid



Test case 1	Total runs	Expected decision	Actual decision	Nodes explored	Execution time	Result
Opening move	10	Corner (1,3,7,9)	Takes corner (Position 1) 10 times.	549945	0.4849 seconds	Correct

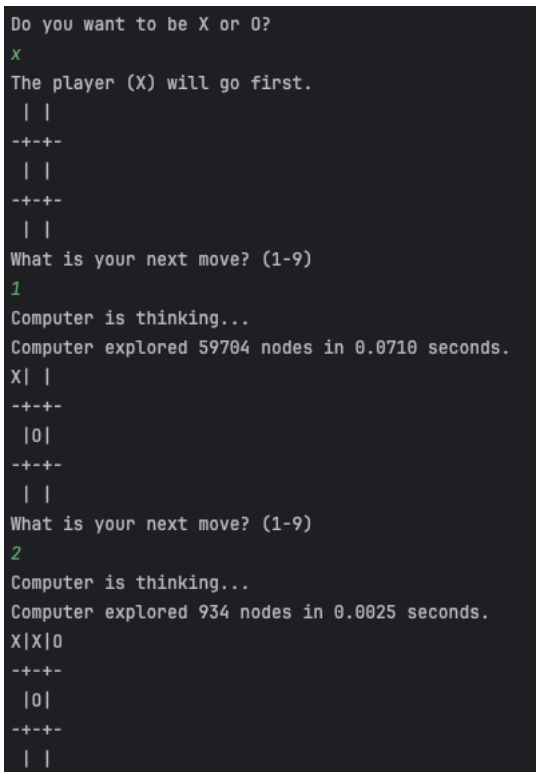
Analysis

1. Strategic Optimality (Correctness)
    - Observation: The agent consistently chose Position 1 (Corner) in all 10 runs.
    - Explanation: In Tic-Tac-Toe, playing a Corner is mathematically proven to be an optimal strategy. The agent searched the entire game tree, evaluated that Position 1 leads to a final utility of 0 (Draw)—which is the best possible outcome against a perfect opponent—and selected it. This confirms the correctness of the utility propagation logic.

2. Deterministic Behaviour
    - Observation: The "Actual Decision" was identical (Position 1) for every single run.
    - Explanation: The Minimax algorithm does not use randomness (unlike a human or a heuristic-based agent). It iterates through moves sequentially (1 to 9). Since Position 1 is the first move checked and it is an optimal move, the algorithm locks it in as the best_move. Even though other corners (3, 7, 9) are equally good, the algorithm sees them as equal to Position 1, not better, so it sticks with the first optimal solution found.

3. Computational Cost (Efficiency Analysis)
    - Observation: The agent explored 549,945 nodes in 0.4849 seconds.
    - Explanation: This high node count illustrates the exponential time complexity ($O(b^m)$) of the algorithm. On an empty board, the branching factor (b) is 9 and the depth (m) is 9. The agent had to generate and evaluate roughly half a million game states just to decide on the very first move.

Figure 12: Blocking move



Test case 2	Total runs	Expected decision	Actual decision	Nodes explored	Execution time	Result
Opponent X has X in position 1 and 2	10	Block and place O at position 3	Block (Position 3) 10 times	934	0.0025 seconds	Correct

Analysis

1. Tactical Accuracy (Correctness)

- Observation: The agent correctly identified the immediate threat posed by the opponent (X) having marks in positions 1 and 2. In all 10 runs, it successfully chose Position 3 to block.
- Explanation: This confirms the Minimisation logic of the algorithm. The agent simulated the future where it did not play at Position 3. In those simulations, the opponent (X) would play at Position 3 on the very next turn, resulting in a Loss (Utility -1). By playing at Position 3, the agent forced the game to continue, preserving the possibility of a Draw (Utility 0). Since 0 > -1, the Minimax algorithm mathematically valued the "Block" higher than any other non-blocking move.

2. Computational Efficiency (Search Space Reduction)

- Observation: The agent explored only 934 nodes in 0.0025 seconds.
- Explanation: This demonstrates the dynamic nature of the algorithm's performance. Compared to the Opening Move, this mid-game move was exponentially faster. With marks already on the board, there are fewer legal moves available to check at each step. The game tree is also significantly shallower because the game is closer to a terminal state (Win, Draw or Loss). This proves that the computational "lag" of Minimax is primarily an early-game issue; as the game progresses, the agent becomes nearly instantaneous.

3. Reliability

- Observation: The agent achieved a 100% success rate (10/10) in blocking the threat.
- Conclusion: This validates that the implementation is robust. It proves that the agent prioritises not losing over other indifferent moves, ensuring it acts rationally even when under pressure.

Task 3.3: Performance analysis

1. Analysis of correctness
   The results from the Blocking test case conclusively validate the implementation's correctness. When the agent was presented with a specific threat (opponent marks in positions 1 and 2), it successfully identified that non-defensive moves would lead to a terminal state of -1 (Loss). By consistently selecting Position 3 to block, the agent demonstrated that it correctly compares utility values, prioritizing a 0 (Draw) path over a loss when a +1 (Win) path is unavailable. This proves the utility propagation logic is functioning as intended.

2. Analysis of efficiency
   The comparison between the 2 test cases highlights the exponential time complexity in the algorithm.
   - High latency opening: In the first test case, the agent had to search the full depth tree, resulting in 549945 node visits.
   - Lower latency mid game: In test case 2, the board was already partially filled. The reduced depth lowered node count to 934, this shows that performance improves the more the game progresses.

3. Effects of parameters and constraints.
   - Search depth: The code currently has unlimited search depth. If depth limit had been set, then test case 1 would have been much faster however the agent would have lost its optimality and fail to detect strategies like the “fork” strategy.
   - Constraints: The 3x3 board constraint is the factor allowing the algorithm to be brute force. If the board size was increased to 4x4 or 5x5, the space would have been exponentially increased and making it not feasible to use a brute force algorithm.

4. Implementation challenges.
   - The opening test case revealed a significant challenge regarding execution overhead. Calculating 549945 nodes for a single move indicates that running the algorithm repeatedly would cause perceptible latency This highlights the need for memoisation to cache these evaluated states and remove redundant calculations in future turns.

   - One other challenge was the difficulty in writing the code, this is because I had not written a minimax algorithm in any programming language before, and it was difficult to translate what I had learnt in lessons into written code. Alternating control between player and agent while ensuring that utility values were propagated correctly also proved tricky and required rigorous debugging. (Swaminathan et al., 2020).

Task 4: Knowledge Representation and Reasoning

Task 4.1: Knowledge representation

State representation:

World state is represented as a 3x3 matrix where each cell (r, c) holds a value from D = {X, O, empty}

$$B_{r,c} \in \{X, O, \emptyset\} \text{ for } r, c \in \{1, 2, 3\}$$

Constraints:

The agent applies constraints to filter invalid actions. A move is only considered if it satisfies

$$\text{Allowed}(Move(r, c)) \iff B_{r,c} = \emptyset$$

Logical relationships (Winning conditions):

A player P wins if they have three of their mark in any row, column, or diagonal.

Function Win(B, P) which returns TRUE if player P has won on board B.

Row wins

A player P wins in row i:

$$\text{Row}(B, P, i) = \bigwedge_{j=0}^{2} (B(i, j) = P)$$

This means B(i, 0) = P AND B(i, 1) = P AND B (i, 2) = P.

Column Wins

A player P wins in column j:

$$\text{Col}(B, P, j) = \bigwedge_{i=0}^{2} (B(i, j) = P)$$

This means B(0, j) = P AND B(1, j) = P AND B (2, j) = P.

Diagonal wins

A player P wins on the main diagonal

$$\text{Diag}_1(B, P) = \bigwedge_{i=0}^{2}(B(i, i) = P)$$

This means B (0, 0) = P AND B (1, 1) = P AND B(2, 2) = P

A player P wins on the anti-diagonal

$$\text{Diag}_2(B, P) = \bigwedge_{i=0}^{2}(B(i, 2 - i) = P)$$

This means B (0, 2) = P AND B (1, 1) = P AND B (2, 0) = P

Final win equation

$$\text{Win}(B, P) = \left(\bigvee_{i=0}^{2} \text{Row}(B, P, i)\right) \vee \left(\bigvee_{j=0}^{2} \text{Col}(B, P, j)\right) \vee \text{Diag}_1(B, P) \vee \text{Diag}_2(B, P)$$

Heuristic knowledge:

The system represents the desirability of a state using a numerical utility function.

$$\textbf{IF } \text{Win}(AI) \textbf{ THEN } U(s) = +1$$

$$\textbf{IF } \text{Win}(Opponent) \textbf{ THEN } U(s) = -1$$

$$\textbf{IF } \neg\text{Win}(AI) \wedge \neg\text{Win}(Opponent) \wedge \text{Full}(B) \textbf{ THEN } U(s) = 0$$

Task 4.2: Reasoning process

The agent employs Adversarial Reasoning via Backward Induction. This system reasons backwards from the future.

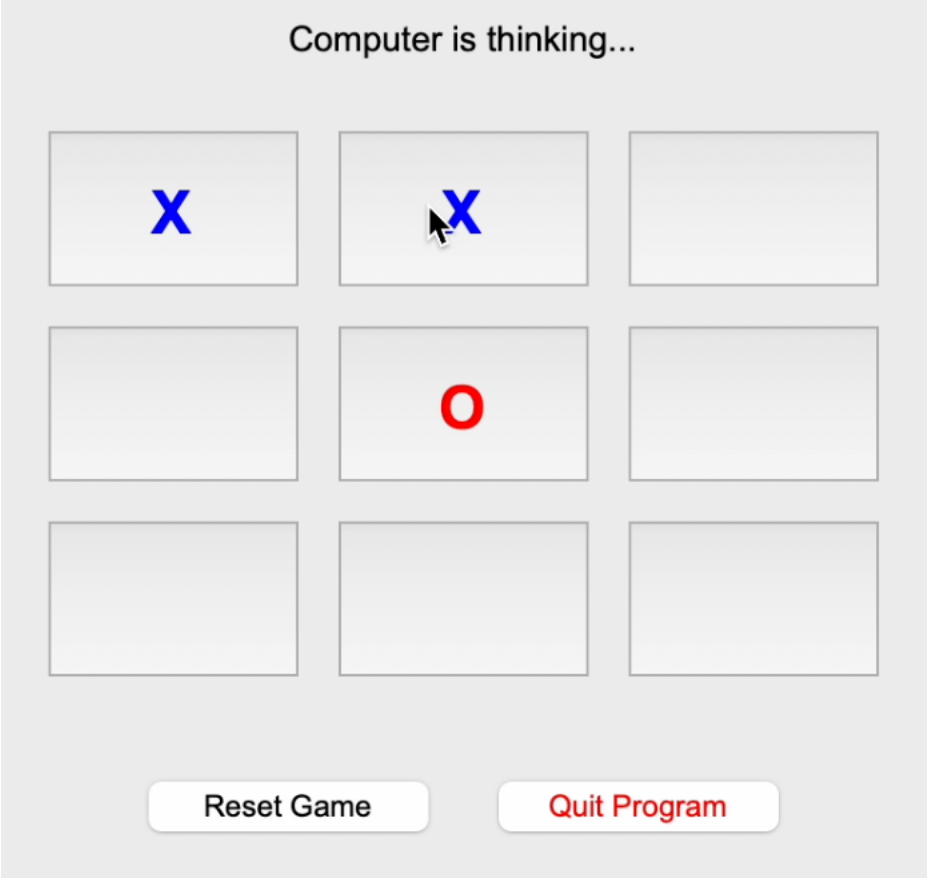
Minimax infers the value of the current state by simulating hypothetical futures.

The agent first considers Move M1. It recursively simulates the opponent's best response to M1. If the opponent's best response leads to a loss(-1) for the agent, the move M1 is inferred as a bad move. The agent then selects another move where the opponent's best response leads to the highest possible utility.

Inference chain

Agent is O and player is X.

Figure 13: Game in progress



The agent first decides whether to play at position 3 or position 6.

Branch A (Simulates playing at position 6):

- Agent plays position 6.
- Opponent's turn: Agent checks that when opponent chooses position 3 it results in a win for the opponent. Win for opponent, utility = -1.
- Prediction: Opponent will play at position 3.
- Branch A value: -1 (loss)

Branch B (Simulates playing at position 3):

- Agent plays position 3.
- Opponent's turn: Agent checks that when opponent chooses position 6, the opponent cannot win immediately.
- Simulation continues: Search ends in a full board state.
- Branch B value: 0 (Draw)

Final deduction: Since 0 > -1, position 3 is the optimal move.

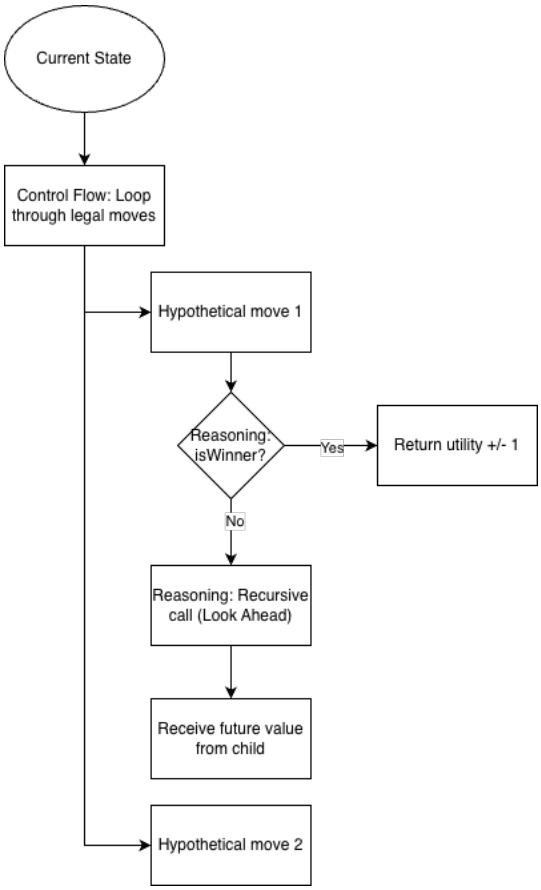
Task 4.3: Integration with algorithm

The recursion stack is tightly integrated into the algorithm's control flow.

- Inference guiding search: The isWinner function acts as the base case. When the reasoning engine detects a Truth value for isWinner, it halts that branch of the search immediately.

- Value propagation: The reasoning result (+1, 0, -1) is passed up the call stack, effectively informing the previous move of its future consequences.

Figure 14: Algorithm integration



Discussion: Advantages and Limitations

Advantages of this representation:

1. Optimality: By representing the problem using formal logic and exhaustive search, the reasoning is always correct. The agent cannot be tricked by strategies used by the player.

2. Completeness: The formal constraints ensure every valid possibility is considered as the agent searches through the entire decision tree.

Limitations:

1. Scalability: This method relies on representing the entire state space. This is only possible after limiting the tic tac toe search space to be 3x3. If it was 4x4 or 5x5, the search space would be too large and take too long to search through all options.

2. Lack of intuition: The agent does not understand different strategies in tic tac toe and only understands terminal values. It derives the concept of different strategies through brute-force calculation rather than understanding the concepts.

Task 5: Evaluation and Ethical Reflection

Task 5.1: Evaluation completeness

To assess the performance of the agent, I evaluated it in 3 distinct environments, agent vs agent, a trial with a experienced player and a trial with a less experienced player.

Task 5.2: Performance analysis

```
Select Mode:
1: Play against the Minimax AI
2: Run AI-vs-AI simulation
Enter 1 or 2: 2
Enter number of games to simulate (e.g., 100): 100
Starting 100 AI-vs-AI games...
  ...Game 10/100 completed.
  ...Game 20/100 completed.
  ...Game 30/100 completed.
  ...Game 40/100 completed.
  ...Game 50/100 completed.
  ...Game 60/100 completed.
  ...Game 70/100 completed.
  ...Game 80/100 completed.
  ...Game 90/100 completed.
  ...Game 100/100 completed.

--- Simulation Complete ---
Total Games: 100
AI 'X' Wins: 0
AI 'O' Wins: 0
Draws:       100
Total Nodes Explored: 61,817,500
Total time taken: 52.1541 seconds
```

Test case	Total runs	Expected decision	Actual decision	Nodes explored	Execution time	Result
100 games (agent vs agent)	10	100% draws	100 draws 10 times	61 817 500	52.1541 seconds	Correct

Analysis: This serves as the control group. Since Tic-Tac-Toe is a solved game, two perfect players must always reach a draw. The 100% draw rate confirms the agent is functioning with mathematical perfection. It successfully identifies and blocks every possible winning attempt by its clone. However, the agent's computations are computationally expensive. This high cost justifies the need for optimizations if the system were to be deployed to weaker hardware or scaled to larger board sizes.

Kevin	Total games	Agent win	Agent loss	Draw
Player X	10	1	0	9
Player O	10	2	0	8

Analysis: The high percentage of draws indicates that this player understands the optimal strategy of Tic-Tac-Toe well. However, the agent was able to capitalize on 3 specific mistakes. This demonstrates that the agent remains vigilant; even against a strong opponent, it maximises its utility immediately when a sub-optimal move is made.

Mother	Total games	Agent win	Agent loss	Draw
Player X	10	5	0	5
Player O	10	5	0	5

Analysis: The win rate spiked significantly to 50%. This suggests that this demographic may be less familiar with the specific strategies of Tic-Tac-Toe or may have a different risk tolerance. The agent successfully exploited these frequent sub-optimal moves. Unlike a human player who might make it easier against a beginner, the agent maximised its score, winning every single game where a mistake allowed it to.

Task 5.3: Ethical reflection

While tic tac toe is a benign topic, the architecture of this agent raises significant ethical considerations.

1. **Fairness, Accessibility and Human-Agent interaction**
   The agent is designed to be "invincible." In a gaming context, this creates a User Experience issue where the agent is not beatable potentially frustrating users when they realise, they cannot win. A responsible agent should perhaps include difficulty scaling to remain accessible and enjoyable.

   However, the user trials revealed a counter-intuitive social dynamic. During testing, participants who lost repeatedly to the agent remained calm and accepting of the result. In contrast, when the same participants lost to an experienced human player, they exhibited negative emotional reactions.

   This discrepancy suggests that users attribute different "intent" to the agent versus humans.

   The agent is perceived as an objective, unfeeling tool—like a calculator. Losing to it is viewed as an inevitability of the system and therefore does not bruise the user's ego. When in comparison, losing to a human involves social hierarchy and competition. The negative reaction to the human opponent stems from the social implications of defeat, whereas the agent exists outside this social contract.

   While the "calm" reaction to the agent is positive in the short term, the lack of emotional engagement is a long-term flaw. A fair and socially responsible agent should mimic a human’s interactions by

occasionally playing sub-optimally to maintain engagement and social enjoyment, rather than ruthlessly maximising its utility function all the time.

2. **Explainability and Transparency**
   The Minimax algorithm is inherently transparent—every decision can be traced back to a specific utility value (+1, -1, 0). However, to a non-technical user, the move is difficult to understand. The agent takes Corner 1 not because of a strategy, but because of a calculation 9 steps ahead. Bridging this gap by having the agent output why it made a move would enhance trust and interpretability.

References

Swaminathan, B., Vaishali, R. and Subashri, T.S.R. (2020) ‘Analysis of minimax algorithm using tic-tac-toe’, *Advances in Parallel Computing* [Preprint]. doi:10.3233/apc200197.